

中山大学计算机学院本科生实验报告

(2024 学年第 1 学期)

课程名称: Data structures and algorithms

任课教师: 张子臻

年级	23 级	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇
电话	13421145433	Email	linhy69@mail2.sysu.deu.cn
开始日期	2024.10.11	完成日期	2024.10.11

第一题

1. 实验题目

明明的随机数

2. 实验目的

明明想在学校中请一些同学一起做一项问卷调查,为了实验的客观性,他先用计算机生成了 N 个 1 到 1000 之间的随机整数 ($N \leq 100$), 对于其中重复的数字, 只保留一个, 把其余相同的数去掉, 不同的数对应着不同的学生的学号。然后再把这些数从小到大排序, 按照排好的顺序去找同学做调查。请你协助明明完成“去重”与“排序”的工作。

3. 算法设计

1. 排序利用 sort 函数

首先利用 C++ STL 中的 sort 函数对序列进行排序。排序的目的是为了将相等的元素排列在一起, 便于后续去重操作。

2. 去重, 在排序后的序列里利用前后对比去重复的元素

排序完成后, 利用循环遍历排序后的序列, 逐一比较相邻元素。如果相邻两个元素相等, 则将当前元素删除。通过 erase 函数删除重复的元素, 使得序列中只保留唯一的元素。

4. 程序运行与测试

1. 排序

```
sort(v1.begin(), v1.end());
```

2.去重

```
for(int i=0;i<v1.size()-1;)
{
    if(v1[i]==v1[i+1])
    {
        v1.erase(v1.begin()+i);
    }
    else
        i++;
}
```

5. 实验总结与心得

1.该题比较简单

该问题的解决方案主要依赖于两个常见操作：排序和去重。这是一个较为基础的算法问题，涉及的操作较为直观，重点在于合理利用现有的 STL 函数。

2.主要解决排序和去重的操作

排序使得重复的元素能够被相邻地排列，而去重则通过遍历相邻元素，删除重复的部分。关键点是 `erase` 函数的使用，它会在删除元素后自动调整序列。

3.算法优化与注意事项

排序的时间复杂度是 $O(n \log n)$ ，去重的时间复杂度是 $O(n)$ 。因此，整体算法的时间复杂度是 $O(n \log n)$ 。使用 `erase` 函数时会导致序列中元素的移动，因此每次删除元素时，时间复杂度为 $O(n)$ ，但对于本题来说，整体时间复杂度依然是 $O(n \log n)$ 。

在处理较大数据量时，可以考虑避免在循环中频繁使用 `erase`，因为这可能会影响效率。

第二题

1. 实验题目

Inversion Number

2. 实验目的

Let $A(1), \dots, A(n)$ be a sequence of n numbers. If $i < j$ and $A(i) > A(j)$, then the pair (i, j) is called an inversion pair.

The inversion number of a sequence is one common measure of its sortedness. Given the sequence A , calculate its inversion number.

3. 算法设计

1. 利用归并计算逆序对

归并排序是一个经典的分治算法，核心思想是将一个大问题分解成多个小问题，逐步解决。利用归并排序的过程，我们可以在合并两个有序子数组的同时，计算逆序对的数量。逆序对的定义是，对于数组中的两个元素 $arr[i]$ 和 $arr[j]$ ，如果 $i < j$ 且 $arr[i] > arr[j]$ ，则称这对元素为逆序对。

2. 合并两个已排序的子数组，并计算逆序对

在归并排序的合并阶段，我们不仅要两个有序的子数组合并成一个有序数组，还需要计算合并过程中产生的逆序对。在合并过程中，如果 $arr[i] > arr[j]$ ，则所有从 i 到 mid 的元素都会与 $arr[j]$ 形成逆序对，因为这部分元素都大于 $arr[j]$ ，而 $i < j$ 。

4. 程序运行与测试

1. // 归并排序并计算逆序对

```
long long mergeSortAndCount(vector<int>& arr, int left, int right) {
    long long invCount = 0;
    if (left < right) {
        int mid = left + (right - left) / 2;

        invCount += mergeSortAndCount(arr, left, mid);
        invCount += mergeSortAndCount(arr, mid + 1, right);
        invCount += mergeAndCount(arr, left, mid, right);
    }
    return invCount;
}
```

2. // 合并两个已排序的子数组，并计算逆序对

```
long long mergeAndCount(vector<int>& arr, int left, int mid, int right) {
    int i = left, j = mid + 1;
    long long invCount = 0;
    vector<int> temp;

    // 合并两个子数组
    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp.push_back(arr[i++]);
        } else {
            temp.push_back(arr[j++]);
            invCount += (mid - i + 1); // 计算逆序对
        }
    }
}
```

```

while (i <= mid) temp.push_back(arr[i++]);
while (j <= right) temp.push_back(arr[j++]);

// 将合并后的结果拷贝回原数组
for (int k = left; k <= right; k++) {
    arr[k] = temp[k - left];
}

return invCount;
}

```

5. 实验总结与心得

1. 算法主要理解归并排序的原理，利用递归将问题化简

归并排序通过递归地将大问题分解成小问题，逐步解决。通过这种方式，我们不仅能够完成排序任务，还可以在合并的过程中同时计算逆序对。理解归并排序的过程是理解该算法的关键。

2. 通过这一道实验题可以加深对归并算法的理解

在归并排序中，合并操作的精髓不仅仅是将两个有序数组合并成一个有序数组，而是要通过比较数组元素的大小来统计逆序对。这一过程能够帮助加深对归并排序的理解，尤其是在排序过程中获取额外的信息（如逆序对）。

3. 归并排序函数中需要搞清楚函数之间的调用关系和逻辑

归并排序的核心在于递归函数的设计。需要注意的是，递归调用 `mergeSortAndCount` 函数时，如何分割左右子数组，以及如何利用 `mergeAndCount` 函数来计算和合并逆序对。在实现时，要确保递归分解和合并的过程是正确的，且计算逆序对的逻辑没有遗漏。

4. 时间复杂度与空间复杂度

时间复杂度： $O(n \log n)$ ，其中 n 是数组的大小。归并排序的时间复杂度是 $O(n \log n)$ ，合并过程的逆序对计算是在线性时间内完成的，因此整体时间复杂度为 $O(n \log n)$ 。

空间复杂度： $O(n)$ ，需要额外的数组来存储合并结果。

应用与优化

归并排序在很多应用场景下表现优异，尤其是在大规模数据排序时，具有稳定的时间复杂度。而逆序对的计算也在很多问题中有应用，比如排序算法的优化、逆序对的统计等。

第三题

1. 实验题目

Mergesort for List

2. 实验目的

Please use mergesort to sort a linked list of data.

The linked list is defined as follows.

```
struct linkedlist{  
  
    int data;  
  
    linkedlist *next;  
  
};
```

Please implement the following function.

```
#include "mergeSort.h"//sort the List by mergesort and return the head of itvoid mergesort(linkedlist *&head, int len){  
  
    //add your code here  
  
}
```

3. 算法设计

1.设计主要函数：利用归并排序对链表进行排序

归并排序的核心思想是分治法，将一个大问题分解成多个小问题，递归地解决。对于链表，归并排序的关键是正确地将链表分割成两部分，并且合并时保持链表结构。排序操作的主要步骤包括：分割链表、递归排序每一部分、合并排序好的部分。

2.设计针对链表需要的合并函数和分割函数

分割函数 (split): 将链表分成两部分。

合并函数 (merge): 将两个已排序的链表合并成一个新的排序链表。

4. 程序运行与测试

1.归并排序函数

```
void mergesort(linkedlist *&head, int len) {  
    if (head == nullptr || head->next == nullptr) {  
        return; // 基础情况：如果链表为空或只有一个节点  
    }  
  
    linkedlist *a;  
    linkedlist *b;  
  
    // 将链表分成 'a' 和 'b' 两部分
```

```

split(head, &a, &b);

// 递归排序每一部分
mergesort(a, len / 2);
mergesort(b, len - len / 2);

// 合并已排序的部分
head = merge(a, b);
}

```

5. 实验总结与心得

1.实验同正常的归并排序相比，难点在于对链表的处理

与普通的数组归并排序相比，链表的归并排序具有额外的复杂性，主要体现在如何对链表进行有效的分割与合并。链表不像数组那样可以直接通过索引访问元素，因此需要使用不同的方法（如快慢指针）来分割链表。

2.仍然需要掌握对归并排序原理的使用

尽管在链表中实现归并排序时需要调整一些细节，但归并排序的基本原理依然适用。理解归并排序的分治法思想，以及如何在分割和合并阶段管理链表指针，是实现链表归并排序的关键。

3.需要根据链表，而不是一般的数组，写出对应的对链表进行处理的函数

由于链表在内存中是分散存储的，无法通过索引直接访问，因此在链表归并排序中，我们需要通过指针操作来完成链表的分割和合并操作。这需要针对链表的特性进行特殊的设计与实现，比如使用 `split` 函数通过快慢指针分割链表、使用 `merge` 函数合并两个有序链表等。

4.时间复杂度与空间复杂度

时间复杂度： $O(n \log n)$ ，其中 n 是链表的节点数。与普通的数组归并排序一样，归并排序的时间复杂度是 $O(n \log n)$ ，主要由于分割和合并操作。

空间复杂度： $O(\log n)$ ，归并排序的递归深度为 $\log n$ ，因此空间复杂度主要来自于递归栈。